# Lecture09: Callback, forEach, map filter, reduce

**The Core Idea:** Instead of manually writing a for loop every time, you tell the array *what* you want to do, and the array method handles the looping for you.

Let's use this sample array for our examples:

```
const products = [
  { id: 1, name: "Laptop", category: "Electronics", price: 1200, inStock: true },
  { id: 2, name: "Book", category: "Books", price: 30, inStock: true },
  { id: 3, name: "Coffee Maker", category: "Appliances", price: 150, inStock:
false },
  { id: 4, name: "Headphones", category: "Electronics", price: 200, inStock: true
}
];
```

### 1. .forEach() - The Simple Looper

**First Thought:** "I want to walk along the conveyor belt and do something with each item, but I'm not creating a new line of items."

**Core Purpose:** To execute a function once for each element in the array. It's a modern alternative to a for loop.

**Key Characteristics:**

It does **not** return anything (it returns undefined).

It does **not** create a new array.

You **cannot** break out of it or continue to the next iteration.

**Syntax:**array.forEach((element, index) ⇒ { /* ... your code ... */ });

element: The current item being processed in the array.

index (Optional): The index of the current item.

**Detailed Example:** Just logging the name of each product.

```
console.log("--- Our Products ---");
products.forEach(product ⇒ {
```

```
  console.log(`- ${product.name}`);
});
```

**Output:**

- -- Our Products ---
  - Laptop
  - Book
  - Coffee Maker
  - Headphones

**When to Use:** When you need to "do something" for each item but you don't need to create a new array from the results. Examples: logging, updating a UI, saving each item to a database.

### 2. .map() - The Transformer

**First Thought:** "I have a list of raw materials. I want to put each one through a machine to create a **new list** of finished products."

**Core Purpose:** To create a **new array** by transforming every element from an original array.

**Key Characteristics:**

It **always** returns a **new array**.

The new array will **always** have the **same length** as the original array.

It is **non-mutating**; it does not change the original array.

**Syntax:**const newArray = array.map((element, index) ⇒ { return /* new value */; });
The return value from your callback function becomes the element in the new array at that same position.

**Detailed Example:** Let's create a new array containing just the names of the products for display.

```
const productNames = products.map(product ⇒ {
  return product.name;
});
```

```
console.log(productNames); // ["Laptop", "Book", "Coffee Maker", "Headphones"]
console.log(products);     // The original `products` array is unchanged!
```

**When to Use:** When you need a new array that is a modified version of the original. This is one of the most-used array methods.

### 3. .filter() - The Sieve / The Bouncer

**First Thought:** "I have a big list of items. I want to run each one through a test and create a **new, shorter list** containing only the items that pass the test."

**Core Purpose:** To create a **new array** containing only the elements from the original array that meet a specific condition.

**Key Characteristics:**

It **always** returns a **new array**.

The new array can have the **same length or be shorter** than the original. It will never be longer.

It is **non-mutating**.

**Syntax:**const newArray = array.filter((element, index) ⇒ { return /* true or false */; });
If your callback function returns true, the element is kept. If it returns false, the element is discarded.

**Detailed Example:** Let's create a new array of only the products that are in stock and are in the "Electronics" category.

```
const availableElectronics = products.filter(product ⇒ {
  // The condition must evaluate to true or false.
  // We combine two conditions here.
  return product.inStock === true && product.category === "Electronics";
});

console.log(availableElectronics);
// [
//   { id: 1, name: "Laptop", ... },
//   { id: 4, name: "Headphones", ... }
// ]
```

**When to Use:** Whenever you need to select a subset of data from an array based on one or more conditions.

### 4. .reduce() - The Accumulator / The Snowball

**First Thought:** "I want to roll up this entire list into a **single final value**." (e.g., Summing a list of numbers, counting items, etc.)

**Core Purpose:** To execute a "reducer" function on each element of the array, resulting in a single output value.

**Key Characteristics:**

It is the most powerful and flexible of the iteration methods.

It can return **any type of value**: a number, a string, an object, another array.

It is **non-mutating**.

**Syntax:**const finalValue = array.reduce((accumulator, currentValue, index) ⇒ { /* ... */ return newAccumulator; }, initialValue);

accumulator: The value that is "accumulated" or carried over from the previous iteration. It's the snowball.

currentValue: The current element being processed.

initialValue: The **starting value** for the accumulator (the small snowball you start with). This is a crucial argument.

**Detailed Example:** Let's calculate the total value of all the items that are currently in stock.

```
const totalStockValue = products.reduce((total, product) ⇒ {
  console.log(`Current Total: ${total}, Current Product: ${product.name}, Price: ${product.price}`);

  if (product.inStock) {
    // The return value of this step becomes the 'total' for the NEXT step.
    return total + product.price;
  }
  // If not in stock, just return the current total without adding anything.
  return total;
}, 0); // Our initial value for the total is 0.
```

```
console.log(`\\nFinal Total Stock Value: $${totalStockValue}`);
```

**Trace of the Output:**

```
Current Total: 0, Current Product: Laptop, Price: 1200
Current Total: 1200, Current Product: Book, Price: 30
Current Total: 1230, Current Product: Coffee Maker, Price: 150
Current Total: 1230, Current Product: Headphones, Price: 200

Final Total Stock Value: $1430
```

**When to Use:** When you need to derive a single summary value from an array. Examples: sum, average, finding the most expensive item, grouping items into an object.

**5. Other Essential Methods for Finding & Testing**

**.find():** Like .filter(), but it **stops** and returns the **very first element** that matches the condition. If nothing matches, it returns undefined.

```
const coffeeMaker = products.find(product ⇒ product.name === "Coffee Maker");
console.log(coffeeMaker); // The coffee maker object
```

**.some():** Checks if **at least one** element in the array passes the test. Returns true or false. It stops as soon as it finds one.

```
const hasOutOfStockItems = products.some(product ⇒ product.inStock === false);
console.log(hasOutOfStockItems); // true
```

**.every():** Checks if **all** elements in the array pass the test. Returns true or false. It stops as soon as it finds one that *doesn't* pass.

```
const areAllItemsInStock = products.every(product ⇒ product.inStock === true);
console.log(areAllItemsInStock); // false
```

## Set and Map

**Part 1: The Set Object**

**First Thought: A Set is a list that enforces uniqueness. It's a collection of items where duplicates are impossible.**

Think of it like a members-only club. You can add a person to the member list, but you can't add the same person twice. The Set automatically handles this for you.

**A. The Basics of Set**

**1. Creating a Set**
You create a Set with new Set(). You can optionally pass in an iterable (like an array) to pre-populate it. Duplicates are automatically removed.

```
// Create an empty Set
const mySet = new Set();

// Create a Set from an array (duplicates are ignored)
const numbersArray = [1, 2, 3, 3, 4, 2, 5];
const numbersSet = new Set(numbersArray);

console.log(numbersSet); // Set(5) { 1, 2, 3, 4, 5 }
```

**2. Core Methods (CRUD)**

**.add(value):** Adds a new element. If the element already exists, it does nothing. It returns the Set object, so you can chain .add() calls.

**.has(value):** Checks if an element exists. Returns true or false.

**.delete(value):** Removes a specific element.

**.clear():** Removes all elements from the Set.

```
const userRoles = new Set();

// Add elements (chaining)
```

```
userRoles.add("editor").add("viewer");
console.log(userRoles); // Set(2) { "editor", "viewer" }

// Add a duplicate - nothing happens
userRoles.add("editor");
console.log(userRoles); // Still Set(2) { "editor", "viewer" }

// Check for an element
console.log(userRoles.has("admin")); // false
console.log(userRoles.has("editor")); // true

// Delete an element
userRoles.delete("viewer");
console.log(userRoles.has("viewer")); // false

// Clear the entire Set
userRoles.clear();
console.log(userRoles); // Set(0) {}
```

**3. The .size Property**

Unlike arrays which have .length, a Set has a .size property to tell you how many items it contains.

```
console.log(numbersSet.size); // 5
```

**B. Iterating Over a Set**

Sets are iterable. The best way to loop over a Set is with a for...of loop. The items are iterated in **insertion order**.

```
const permissions = new Set(["read", "write", "execute"]);

for (const permission of permissions) {
  console.log(permission);
}
// Outputs:
// read
```

```
// write
// execute
```

You can also use .forEach().

**C. Real-World Use Cases (Why use a Set?)**

**Removing Duplicates from an Array (The #1 Use Case):** This is a powerful and concise one-liner.

```
const duplicateEmails = ["a@a.com", "b@b.com", "a@a.com"];

// Convert to a Set to remove duplicates, then spread it back into a new array.
const uniqueEmails = [...new Set(duplicateEmails)];

console.log(uniqueEmails); // ["a@a.com", "b@b.com"]
```

**Checking for Uniqueness / Existence:** Set.has() is much faster than Array.includes() for very large datasets. If you just need to track if you've "seen" an item before, a Set is far more performant.

```
// Imagine tracking unique visitors to a page
const visitedUsers = new Set();

function userVisits(userId) {
  if (!visitedUsers.has(userId)) {
    console.log(`Welcome, new visitor #${userId}!`);
    visitedUsers.add(userId);
  } else {
    console.log(`Welcome back, visitor #${userId}!`);
  }
}

userVisits(101); // Welcome, new visitor #101!
userVisits(102); // Welcome, new visitor #102!
userVisits(101); // Welcome back, visitor #101!
```

| Use... | When you need... |
| --- | --- |
| **Array** | An ordered list where **duplicates are allowed** and you need index-based access (arr[0]). |
| **Set** | A collection of **unique values** where the main task is adding, deleting, and checking for existence. |

**Part 2: The Map Object**

**First Thought: A Map is like an object, but its keys can be *anything* (not just strings).**

This is the most critical difference. In a standard object {}, keys are automatically converted to strings. A Map preserves the key's type, allowing you to use objects, functions, or any other data type as a key.

**A. The Basics of Map**

**1. The Problem with Plain Objects**

```
let myObject = {};
let keyObject1 = { id: 1 };
let keyObject2 = { id: 2 };

// Both keyObject1 and keyObject2 get converted to the SAME string: "[object Object]"
myObject[keyObject1] = "Value for key 1";
myObject[keyObject2] = "Value for key 2";

// The second assignment overwrote the first!
console.log(myObject); // { "[object Object]": "Value for key 2" }
```

**2. Creating a Map**

You create a Map with new Map(). You can pre-populate it with an array of [key, value] pairs.

```
// Create an empty Map
const myMap = new Map();

// Create a Map with initial values
```

```
const userMap = new Map([
  ["name", "Alice"],
  [true, "is verified"],
  [100, "points"]
]);
```

**3. Core Methods (CRUD)**

**.set(key, value):** Adds or updates a key-value pair. Returns the Map, so you can chain it.

**.get(key):** Retrieves the value for a given key. Returns undefined if the key doesn't exist.

**.has(key):** Checks if a key exists. Returns true or false.

**.delete(key):** Removes a key-value pair.

**.clear():** Removes all key-value pairs.

```
const metadata = new Map();
let user1 = { name: "Alice" };
let user2 = { name: "Bob" };

// Set values using objects as keys
metadata.set(user1, { lastLogin: "2023-10-27" });
metadata.set(user2, { lastLogin: "2023-10-26" });

// Get a value using the exact same object reference
console.log(metadata.get(user1)); // { lastLogin: "2023-10-27" }

console.log(metadata.has(user2)); // true
```

**4. The .size Property**

Like Set, a Map has a .size property.

```
console.log(metadata.size); // 2
```

**B. Iterating Over a Map**

A Map is also iterable. The best way to loop is with for...of and **destructuring**. The items are iterated in **insertion order**.

```
const userMap = new Map([
  ["name", "Alice"],
  ["age", 30]
]);

// Use destructuring to unpack the [key, value] pair
for (const [key, value] of userMap) {
  console.log(`${key} → ${value}`);
}
// Outputs:
// name → Alice
// age → 30
```

You can also use .keys(), .values(), and .entries() to get iterators, or .forEach().

**C. Real-World Use Cases (Why use a Map?)**

**Storing Metadata for Objects:** This is the killer use case. You can "attach" data to an object without actually modifying the object itself. This is great for keeping DOM elements clean.

```
const elementData = new Map();
const button1 = document.querySelector("#btn1");

// Associate some metadata with this specific button element
elementData.set(button1, { clicks: 0, lastClickTime: null });

// Now you can store and retrieve data about `button1` without
// ever doing `button1.mydata = ...`, which pollutes the DOM object.
```

**High-Performance Caching:** If you have a function that performs a complex calculation, you can use a Map to cache the results. The arguments to the function can be the key, and the result can be the value.

**When You Need a "Dictionary" with Non-String Keys:** Any time your keys are not simple strings, a Map is the correct and only choice.

| Use... | When you need... |
| --- | --- |
| **Object** | A simple, fixed set of **string-based keys**. Great for storing the structure of a single entity (like a user). |
| **Map** | A collection of key-value pairs where **keys can be of any type**, you need to iterate in insertion order, or you need to get the size quickly. |

### Part 3: Advanced - WeakSet and WeakMap

**First Thought: A "weak" version is a Map or Set that doesn't prevent its items from being garbage collected.**

A normal Map or Set holds a **strong reference** to its items. As long as an object is in a Map, it will **never be garbage collected**.

A WeakMap or WeakSet holds a **weak reference**. If an object is the key in a WeakMap and it's the *only* reference left in your entire program, the garbage collector is free to destroy it, and it will be automatically removed from the WeakMap.

**Use Case:** Caching. In the DOM element example above, if you remove button1 from the page, you want its metadata in the elementData map to be automatically deleted to prevent a **memory leak**. If you used a Map, the data would stay there forever. If you use a WeakMap, it cleans itself up.

**Limitations:**

You can only store **objects** in a WeakSet or as keys in a WeakMap (not primitives).

You **cannot iterate** over them (because the items could disappear at any moment during the loop). They only have .has(), .get(), .set(), and .delete().